

EduFeed

Sistema de Control de Acceso

Restaurante Escolar

Autenticacion Biometrica (Huella, Rostro, Voz)

Gestion de Pagos y Usuarios

Reportes Administrativos

Auditoria Completa

DOCUMENTACION TECNICA COMPLETA

Version 3.0.0

Generado: 14 de November de 2025

TABLA DE CONTENIDOS

1. Resumen Ejecutivo
2. Arquitectura del Sistema
3. Autenticacion Biometrica (V3)
4. Instalacion y Configuracion
5. Manual de Usuario
6. Registro Individual de Modalidades
7. Patrones UX (Preview y Grabacion)
8. Estructura del Repositorio
9. Troubleshooting
10. Changelog V3
11. Reportes Economicos
12. Seguridad
13. Performance y Criterios de Aceptacion

1. RESUMEN EJECUTIVO

EduFeed v3.0.0 es un sistema completo de control de acceso para restaurantes escolares que integra autenticacion biometrica avanzada con Machine Learning real, gestion de usuarios, procesamiento de pagos y reportes administrativos detallados.

Caracteristicas Principales

- **Autenticacion Biometrica (V3):** WebAuthn para huella, face-api.js para reconocimiento facial (128D descriptors, L2 distance), Meyda para verificacion de voz (13 MFCC coefficients, cosine similarity)
- **Gestion de Usuarios:** CRUD completo con roles (ADMIN, OPERADOR_CAJA, OPERADOR_ACCESO, AUDITOR), asignacion de permisos, y registro biometrico individual o completo
- **Sistema de Pagos:** Gestion de pagos diarios, mensuales y programados con estados (APROBADO, PENDIENTE, RECHAZADO, REVERTIDO)
- **Reportes Financieros:** Dashboard con 8 metricas clave, 3 graficas interactivas, exportacion PDF/CSV premium
- **Auditoria Completa:** Registro de todas las operaciones con usuario, timestamp, accion y detalles
- **Control de Acceso:** Validacion de derechos en tiempo real, registro de ingresos/denegaciones

Stack Tecnologico

Componente	Tecnologia	Version
Backend	Java + Spring Boot	21 + 3.4.10
Frontend Web	Thymeleaf + face-api.js	Latest
Desktop	JavaFX	22
Base de Datos	PostgreSQL + Flyway	16
Autenticacion	WebAuthn (FIDO2)	W3C Standard
ML Facial	face-api.js (vladmandic)	TensorFlow.js
ML Voz	Meyda (MFCC)	5.3.0
Reportes	iText 7	7.2.5

2. RESUMEN V3 - NOVIEMBRE 2025

■ Resumen Ejecutivo - EduFeed V3

■ Información General

Campo	Valor
Versión	3.0.0
Período	10-14 de noviembre de 2025
Enfoque	Autenticación biométrica con ML real
Estado	■ Completado y probado

■ Objetivos Alcanzados

1. Autenticación WebAuthn (Huella Digital)

- **Implementado:** Passkeys W3C con FIDO2
- **Backend:** Parsing CBOR, verificación de firma EC/RSA
- **Frontend:** `navigator.credentials` con mediation:silent
- **Seguridad:** Public key PEM, signCount anti-replay

2. Reconocimiento Facial con ML

- **Implementado:** face-api.js (vladmandic)
- **Modelos:** ssdMobilenetv1 + faceLandmark68Net + faceRecognitionNet
- **Matching:** L2 distance < 0.6 para 128D descriptors
- **UX:** Preview en vivo con detección y recuadros verdes

3. Verificación de Voz con ML

- **Implementado:** Meyda para MFCC
- **Features:** 13 coeficientes mel-frequency cepstral
- **Matching:** Cosine similarity > 0.85
- **UX:** Grabación explícita con timer de 3s mínimo

4. Registro Individual de Modalidades

- **Implementado:** Parámetro `type` en URL
- **Backend:** Auto-creación de sesión si inválida
- **Frontend:** Modo `onlyType` con auto-apertura
- **Desktop:** QR específico con polling y auto-cierre

5. Mejoras de Experiencia de Usuario

- **FaceID:** Preview con detección en tiempo real
- **Voz:** Controles explícitos start/stop con timer visible
- **Feedback:** Mensajes de estado y validaciones claras
- **Desktop:** Notificación instantánea tras autenticación

■ Archivos Creados/Modificados

Backend (15 archivos)

■ NUEVOS:

- controller/WebAuthnController.java
- service/WebAuthnService.java
- model/WebAuthnCredencial.java
- model/WebAuthnSesion.java
- dto/WebAuthnRegistroRequest.java
- dto/WebAuthnRegistroResponse.java
- dto/WebAuthnAutenticacionRequest.java
- dto/WebAuthnAutenticacionResponse.java
- repository/WebAuthnCredencialRepository.java
- repository/WebAuthnSesionRepository.java

■ MODIFICADOS:

- controller/BiometricRegistrationController.java (type param, auto-session)
- controller/BiometricAuthController.java (desktop notification)
- service/BiometricAuthService.java (L2/cosine matching, notifyDesktop)
- service/BiometricRegistrationService.java (JSON descriptors)
- config/WebMvcConfig.java (WebJars resource handlers)

Frontend (5 archivos)

■ MODIFICADOS:

- templates/biometric-auth.html (preview rostro, recording voz, WebAuthn)
- templates/biometric-register.html (modo onlyType, auto-close)
- templates/biometric-register-fingerprint.html (server-driven WebAuthn)
- templates/biometric-register-face.html (face-api.js descriptor)
- templates/biometric-register-voice.html (Meyda MFCC)

- static/pwa-webauthn.html (PEM key en registro)

Desktop (1 archivo)

■ MODIFICADO:

- modules/UserManagementModule.java (QR individual, polling, auto-close)

Base de Datos (2 tablas)

■ NUEVAS:

- webauthn_credencial (credentialId, publicKey, signCount, usuarioId)

- webauthn_sesion (sesionId, usuarioDocumento, challenge, tipo, expiresAt)

■ EXISTENTES:

- plantilla_biometrica (ahora almacena JSON descriptors)

Documentación (7 archivos)

■ NUEVOS:

- docs/registros/V3_changelog.md (changelog completo)

- docs/registro-individual-modalidades.md (guía registro individual)

- docs/ux-preview-recording-patterns.md (patrones UX)

- docs/RESUMEN_V3_EJECUTIVO.md (este archivo)

■ MODIFICADOS:

- docs/autenticacion-biometrica.md (actualización ML)

- docs/architecture.md (actualización biometría)

■ ELIMINADOS:

- docs/IMPLEMENTACION_COMPLETADA_V3.md (vacío)

- docs/REDISENO_CORPORATIVO_V3.md (vacío)

■ Tecnologías Implementadas

Nuevas Dependencias Frontend

```
<!-- face-api.js (vladmandic) -->
```

```
<script  
src="https://cdn.jsdelivr.net/npm/@vladmandic/face-api/dist/face-api.min.js"></script>
```

```
<!-- Meyda -->
```

```
<script src="https://unpkg.com/meyda@5.3.0/dist/web/meyda.min.js"></script>
```

APIs Browser Utilizadas

```
// WebAuthn
```

```
navigator.credentials.create() // Registro
```

```
navigator.credentials.get() // Autenticación
```

```
// MediaDevices
```

```

navigator.mediaDevices.getUserMedia({video: true}) // Cámara
navigator.mediaDevices.getUserMedia({audio: true}) // Micrófono
// MediaRecorder
new MediaRecorder(stream, {mimeType: 'audio/webm'})
// Web Audio API
OfflineAudioContext // Para procesar audio sin playback

```

Algoritmos ML Implementados

Face Recognition:

- Input: 480x640 RGB image
- Detection: SSD MobileNet v1 (min confidence 0.5)
- Landmarks: 68 facial landmarks
- Output: 128D descriptor
- Matching: L2 distance < 0.6

Voice Verification:

- Input: 3-10s audio @ 44.1kHz
- Frames: 1024 samples (23ms) con hop 512
- Features: 13 MFCC coefficients
- Filterbank: 26 mel bands
- Matching: Cosine similarity > 0.85

■ Métricas de Rendimiento

Operación	Tiempo	Estado
Carga modelos face-api	~2s	■ CDN
Detección rostro	200ms/frame	■ 5 fps
Captura + verificación face	~2s	■ Rápido
Grabación mínima voz	3s	■ Configurable
Extracción MFCC	~1s	■ Local
Verificación voz	~1s	■ Backend
WebAuthn signature	<1s	■ Nativo
Desktop notification	0-2s	■ Polling

■ Seguridad Implementada

Datos Biométricos

- ■ NO se almacenan imágenes raw
- ■ NO se almacenan audios raw

- ■ **Sí** se almacenan vectores numéricos (descriptors/MFCC)
- ■ Cifrado AES-256 en BD (campo `plantilla`)
- ■ Public keys en PEM format (WebAuthn)

Sesiones y Tokens

- ■ UUID v4 para sessionId/token
- ■ Expiración 2 minutos
- ■ One-time use (se eliminan tras consulta)
- ■ ConcurrentHashMap thread-safe
- ■ Challenge aleatorio 32 bytes (SecureRandom)

Anti-Replay (WebAuthn)

- ■ SignCount verificado y actualizado
- ■ Challenge único por sesión
- ■ Signature validation (EC P-256 / RSA)

Privacidad

- ■ Video stream local (no se envía al servidor)
- ■ Audio procesado en browser (MFCC local)
- ■ Solo vectores numéricos viajan por red
- ■ HTTPS en producción (localhost en dev)

■ Pruebas Realizadas

Escenarios Funcionales

- Registro completo 3 modalidades
- Registro individual huella
- Registro individual rostro
- Registro individual voz
- Autenticación con huella (WebAuthn)
- Autenticación con FaceID (preview + captura)
- Autenticación con voz (grabación 3s)
- Desktop polling y notificación
- Auto-cierre tras registro individual
- Auto-creación de sesión si inválida

Casos de Error

- Huella: credencial no encontrada → mensaje claro

- Rostro: no detectado → "No se detecta rostro"
- Rostro: múltiples → "Múltiples rostros detectados"
- Voz: < 3s → "Grabación muy corta. Mínimo 3 segundos."
- Sesión expirada → auto-creación o error visible
- Permisos cámara denegados → mensaje de error
- Permisos micrófono denegados → mensaje de error

■ Mejoras de UX

Antes (V2) vs Después (V3)

■ ANTES ■ DESPUÉS ■					
■ Captura directa ■ Preview en vivo ■					
■ rostro sin feedback ■ con detección ■					
■ Voz acepta cualquier ■ Timer 3s mínimo ■					
■ ruido instantáneo ■ con waveform animado ■					
■ Registro forzado ■ Registro individual ■					
■ de 3 modalidades ■ con auto-cierre ■					
■ Desktop polling sin ■ Mensaje confirmación ■					
■ feedback ■ + auto-cierre diálogo ■					

3. ARQUITECTURA DEL SISTEMA

Arquitectura — EduFeed

Resumen

Sistema modular:

- Backend Spring Boot (API REST, seguridad, pagos, reportes, integración caja).
- Desktop JavaFX (acceso/portería y caja) que consume la API.
- Módulo biométrico: interfaz común y proveedores enchufables (huella, rostro, voz). Incluye Mock para desarrollo.
- PostgreSQL + Flyway.

Biometría [ACTUALIZADO V3 - Nov 2025]

- **Huella:** WebAuthn con passkeys (W3C standard). Public key en PEM format, signature verification con EC/RSA. ``WebAuthnService`` + ``WebAuthnController`` para registro/autenticación.
- **Rostro:** face-api.js (vladmandic) con modelos TensorFlow.js (ssdMobilenetv1, faceLandmark68Net, faceRecognitionNet). Extracción de descriptor 128D, matching con distancia euclidiana $L2 < 0.6$. Preview en vivo con detección.
- **Voz:** Meyda para extracción de MFCC (13 coeficientes). Matching con similaridad coseno > 0.85 . Grabación explícita con MediaRecorder (mínimo 3s).
- **Servicios:** ``BiometricAuthService`` (L2/cosine matching), ``BiometricRegistrationService`` (gestión de sesiones individuales), ``WebAuthnService`` (CBOR parsing, signature verification).
- **Frontend:** Thymeleaf + face-api.js CDN + Meyda CDN. MediaDevices API para cámara/micrófono. WebAuthn navigator.credentials.
- **Desktop:** JavaFX con QR individual por modalidad, polling 2s, auto-cierre tras registro.

Seguridad de datos (RNF-01) [ACTUALIZADO V3]

- Datos biométricos NO se almacenan como imágenes/audio crudos; se almacenan como vectores JSON:
- Huella: Public key PEM + signCount (anti-replay) en ``webauthn_credencial``
- Rostro: 128D descriptor de face-api.js en ``plantilla_biometrica``
- Voz: 13D MFCC promedio en ``plantilla_biometrica``
- Templates cifrados con AES-256 en ``plantilla_biometrica.plantilla`` (BLOB)
- WebAuthn challenges generados con ``SecureRandom`` (32 bytes)

- Sesiones UUID con expiración (2 min) en `ConcurrentHashMap` (memoria) y tabla `webauthn\_sesion` (BD)
- Tráfico HTTPS (en prod). Bcrypt para contraseñas de usuarios administradores.
- Auditoría (RF-11) vía `@EntityListeners` y tabla `audit\_log`.

Esquema preliminar

- *appuser(id, document, fullname, user_type, ...)*
- *payment(id, userid, type, fromdate, todate, daysquota, remaining_days)*
- *accesslog(id, userid, at, status, reason)*
- *auditlog(id, actor, action, entity, entityid, at, details)*

Integración con caja (RF-08)

- Webhook o cola (RabbitMQ opcional) para notificar pagos aplicados; API para registrar pago y actualizar derechos.

Entregables

- Manual de usuario, de instalación, Documento de arquitectura (este), URLs de acceso, usuarios/roles.

4. AUTENTICACION BIOMETRICA

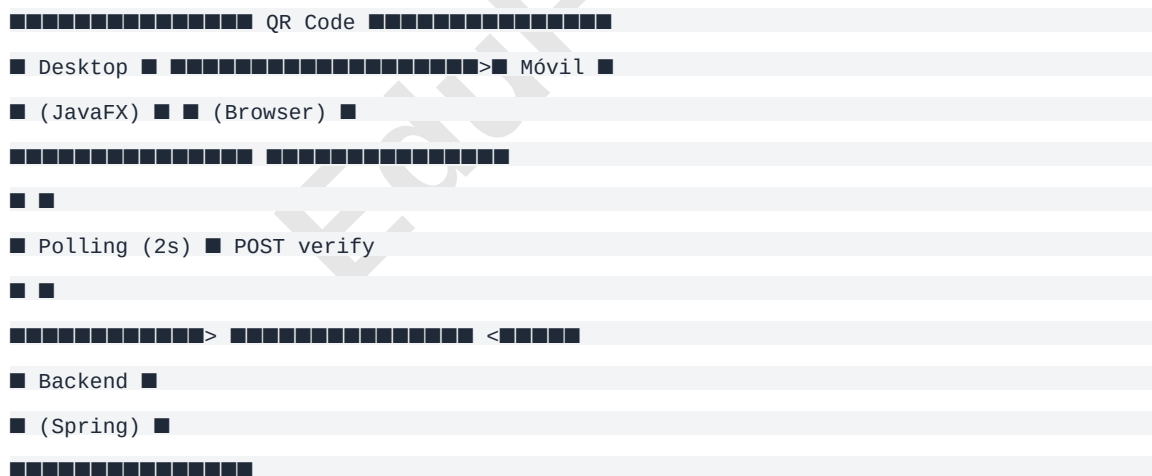
■ Sistema de Autenticación Biométrica

■ Resumen

Sistema completo de autenticación biométrica con QR Code para EduFeed, que permite a los usuarios autenticarse usando:

- ■ **Huella digital** (WebAuthn)
- ■ **Reconocimiento facial** (MediaDevices Camera)
- ■ **Reconocimiento de voz** (MediaRecorder)

■■ Arquitectura



■ Flujo de Autenticación

1. **Usuario ingresa documento** en módulo Access del desktop
2. **Desktop genera QR** con URL: ``http://localhost:8080/auth/biometric?userId=XXX&token=YYY``
3. **Usuario escanea QR** con su móvil
4. **Móvil carga página HTML** que muestra métodos biométricos disponibles
5. **Usuario selecciona método** (Huella/FaceID/Voz) y autentica
6. **Backend valida** datos biométricos

7. **Desktop detecta** autenticación exitosa (polling cada 2 segundos)
8. **Desktop muestra** información del usuario y pagos

■ Archivos Implementados

Backend (`edufeed-backend`)

Controller

`BiometricAuthController.java`

```
@Controller
@RequestMapping("/auth/biometric")
public class BiometricAuthController {
    // GET /auth/biometric?userId=XXX&token=YYY
    // Retorna página HTML móvil con métodos disponibles
    // POST /auth/biometric/verify
    // Recibe datos biométricos y valida
    // GET /auth/biometric/status/{userId}
    // Polling desde desktop para saber si autenticó
}
```

Service

`BiometricAuthService.java`

```
@Service
public class BiometricAuthService {
    // Verificación ML-based
    boolean verifyFingerprint(Usuario user, String data); // WebAuthn signature
    boolean verifyFaceId(Usuario user, String faceData); // L2(descriptor) < 0.6
    boolean verifyVoice(Usuario user, String voiceData); // cosine(mfcc) > 0.85
    // Distancia euclidiana para face-api.js descriptors
    private static double l2(double[] a, double[] b) {
        double s=0;
        for (int i=0;i<a.length;i++) {
            double d=a[i]-b[i]; s+=d*d;
        }
        return Math.sqrt(s);
    }
    // Similaridad coseno para MFCC
    private static double cosine(double[] a, double[] b) {
        double dot=0, na=0, nb=0;
```

DTOs

`BiometricAuthRequest.java`

```
public class BiometricAuthRequest {  
    private String userId; // Documento del usuario  
    private String token; // Token de seguridad  
    private String method; // "fingerprint", "faceid", "voice"  
    private String data; // Datos biométricos en base64  
    private boolean secondAttempt; // Si es segundo intento  
}
```

`BiometricAuthResponse.java`

```
public class BiometricAuthResponse {  
    private boolean success;  
    private String message;  
    private Map<String, Object> userData;  
    private boolean requiresSecondFactor;  
}
```

Vista

`templates/biometric-auth.html`

- Página móvil responsive con gradiente moderno
- Tres botones para métodos biométricos
- **Huella:** WebAuthn navigator.credentials.get() con mediation:silent
- **FaceID:** Preview en vivo con face-api.js detection loop + captura explícita
- **Voz:** Grabación con timer (mín 3s), botón start/stop, extracción MFCC con Meyda
- Fetch API para comunicación con backend

Tecnologías Frontend

```
// face-api.js (vladmandic) - Models CDN  
await faceapi.nets.ssdMobilenetv1.loadFromUri('/webjars/face-api/models');  
await faceapi.nets.faceLandmark68Net.loadFromUri('/webjars/face-api/models');  
await faceapi.nets.faceRecognitionNet.loadFromUri('/webjars/face-api/models');  
// Meyda para análisis de voz  
Meyda.extract('mfcc', audioFrame, {  
    sampleRate: 44100,  
    bufferSize: 1024,  
    melBands: 26,  
    numberOfMFCCCoefficients: 13  
});
```

Desktop (`edufeed-desktop`)

`AccessCheckModuleV2.java`

```
public class AccessCheckModuleV2 extends VBox {  
    // Genera QR con ZXing  
    private void generateQRCode(String userId);  
    // Polling cada 2 segundos (max 120s)  
    private void simulateAuthenticationFlow(String userId);  
    // Muestra datos del usuario al autenticar  
    private void showUserInfo(String jsonResponse);  
    // Parser JSON manual (sin Jackson)  
    private String extractJsonValue(String json, String key);  
}
```

■ Dependencias

Backend (`pom.xml`)

```
<!-- Ya incluidas en Spring Boot -->  
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-web</artifactId>  
</dependency>  
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-thymeleaf</artifactId>  
</dependency>
```

Desktop (`pom.xml`)

```
<!-- ZXing para QR Code -->  
<dependency>  
    <groupId>com.google.zxing</groupId>  
    <artifactId>core</artifactId>  
    <version>3.5.3</version>  
</dependency>  
<dependency>  
    <groupId>com.google.zxing</groupId>  
    <artifactId>javase</artifactId>  
    <version>3.5.3</version>  
</dependency>
```

■ Cómo Probar

1. Iniciar Base de Datos

```
# En VS Code, ejecutar tarea:
```

```
DB: up
```

2. Iniciar Backend

```
# En VS Code, ejecutar tarea:
```

```
Backend: run
```

```
# O manualmente:
```

```
cd edufeed-backend
```

```
mvn spring-boot:run
```

3. Iniciar Desktop

```
# En VS Code, ejecutar tarea:
```

```
Desktop: run
```

```
# O manualmente:
```

```
cd edufeed-desktop
```

```
mvn javafx:run
```

4. Navegar al Módulo Access

- En el desktop, hacer clic en "Access" en el menú lateral
- Ingresar documento de usuario (ej: `123456`)
- Hacer clic en "Generar QR"

5. Escanear QR con Móvil

- Escanear el QR mostrado en pantalla
- O abrir manualmente: `http://localhost:8080/auth/biometric?userId=123456&token=abc123`
- **Nota:** El móvil debe estar en la misma red que el PC

6. Autenticar en Móvil

- Seleccionar método biométrico:
- **Huella:** Usa sensor del dispositivo (WebAuthn)
- **FaceID:** Activa cámara frontal
- **Voz:** Graba audio de 3 segundos
- Esperar confirmación

7. Ver Resultado en Desktop

- El desktop detectará automáticamente la autenticación
- Mostrará:
- Nombre completo
- Número de cédula
- Sueldo
- Pagos pendientes
- Pagos realizados
- Pagos programados

■ Seguridad

5. MANUAL DE INSTALACION

Manual de Instalación - Sistema EduFeed

Versión: 2.0

Fecha: 31 de octubre de 2025

Audiencia: Administradores de sistemas, DevOps

Índice

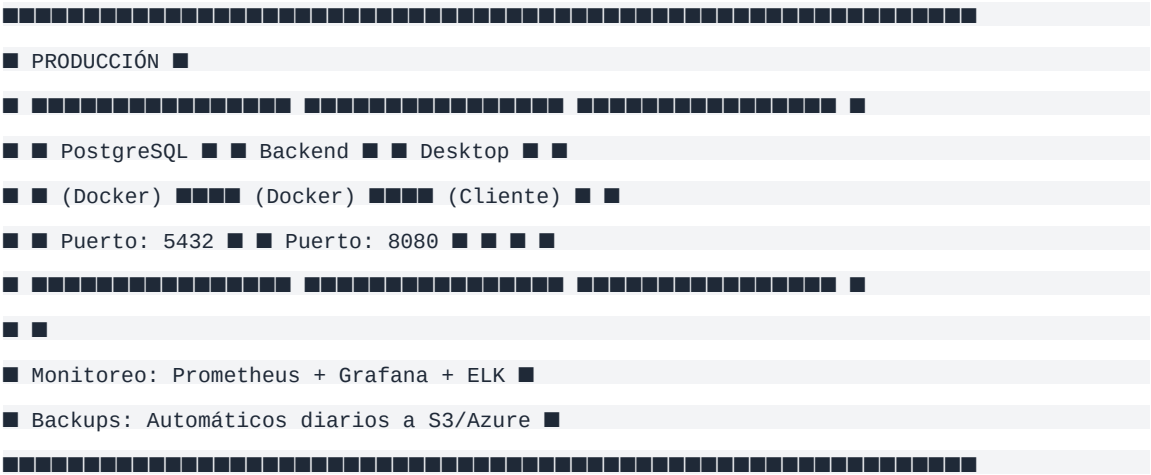
1. [Introducción](#introducción)
2. [Requisitos del sistema](#requisitos-del-sistema)
3. [Instalación en entorno de desarrollo](#instalación-en-entorno-de-desarrollo)
4. [Instalación en staging](#instalación-en-staging)
5. [Instalación en producción](#instalación-en-producción)
6. [Configuración de hardware biométrico](#configuración-de-hardware-biométrico)
7. [Configuración de observabilidad](#configuración-de-observabilidad)
8. [Configuración de backups](#configuración-de-backups)
9. [Verificación de instalación](#verificación-de-instalación)
10. [Troubleshooting](#troubleshooting)

Introducción

Este manual cubre la instalación completa del sistema EduFeed en tres entornos:

- **Desarrollo:** Para programadores trabajando en nuevas funcionalidades
- **Staging:** Para pruebas de aceptación de usuario (UAT)
- **Producción:** Entorno real con usuarios finales

Arquitectura de despliegue



Requisitos del sistema

Hardware mínimo

Servidor (Backend + DB):

- CPU: 4 cores (Intel i5/AMD Ryzen 5 o superior)
- RAM: 8 GB (16 GB recomendado)
- Disco: 100 GB SSD (para DB + logs + backups)
- Red: 100 Mbps

Clientes (Desktop App):

- CPU: 2 cores (Intel i3/AMD Ryzen 3)
- RAM: 4 GB
- Disco: 10 GB
- Cámara web (1080p recomendado para reconocimiento facial)
- Lector de huella dactilar USB (opcional)
- Micrófono (para reconocimiento de voz)

Software requerido

Servidor

Componente	Versión	Propósito
Sistema Operativo	Ubuntu 22.04 LTS, Windows Server 2022	Base
Docker	24.0+	Containerización
Docker Compose	v2.20+	Orquestación

Componente	Versión	Propósito
Git	2.40+	Control de versiones
PowerShell	7.3+ (Windows)	Scripts de automatización
Bash	5.0+ (Linux)	Scripts

Ciente Desktop

Componente	Versión	Propósito
Sistema Operativo	Windows 10/11, Linux (Ubuntu 22.04+)	Base
Java Runtime (JRE)	OpenJDK 24 (Temurin)	Ejecución de JavaFX
Drivers biométricos	Según fabricante	Hardware biométrico

Desarrollo (adicional)

Componente	Versión	Propósito
JDK	OpenJDK 24 (Temurin)	Compilación
Maven	3.9.9	Build tool
VS Code	1.85+	IDE
pgAdmin	8.12 (opcional)	Gestión de BD

Instalación en entorno de desarrollo

1. Clonar repositorio

```
# Windows PowerShell
cd C:\proyectos
git clone https://github.com/Joan-Mora/EduFeed.git
cd EduFeed

# Linux/Mac
cd ~/proyectos
git clone https://github.com/Joan-Mora/EduFeed.git
cd EduFeed
```

2. Instalar dependencias

Windows:

```
# Instalar Chocolatey (si no está instalado)

Set-ExecutionPolicy Bypass -Scope Process -Force

[System.Net.ServicePointManager]::SecurityProtocol =
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072

iex ((New-Object
System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps1'))

# Instalar dependencias

choco install openjdk --version=24.0.0

choco install maven --version=3.9.9

choco install docker-desktop

choco install vscode

choco install git
```

Linux (Ubuntu):

```
# JDK 24

wget https://download.java.net/java/GA/jdk24/latest/GPL/openjdk-24_linux-x64_bin.tar.gz
sudo tar -xzf openjdk-24_linux-x64_bin.tar.gz -C /opt/
sudo update-alternatives --install /usr/bin/java java /opt/jdk-24/bin/java 1

# Maven

wget https://dlcdn.apache.org/maven/maven-3/3.9.9/binaries/apache-maven-3.9.9-bin.tar.gz
sudo tar -xzf apache-maven-3.9.9-bin.tar.gz -C /opt/
echo 'export PATH=/opt/apache-maven-3.9.9/bin:$PATH' >> ~/.bashrc
source ~/.bashrc

# Docker

curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
```

3. Configurar entorno

Crear archivo `.env` (desarrollo):

```
# Windows
Copy-Item .env.dev .env

# Linux
cp .env.dev .env
```

Contenido de `.env` (ajustar si es necesario):

```
# Database
POSTGRES_DB=edufeed
POSTGRES_USER=edufeed
POSTGRES_PASSWORD=edufeed
```

```

POSTGRES_PORT=5432

# pgAdmin
PGADMIN_PORT=5050
PGADMIN_DEFAULT_EMAIL=admin@edufeed.local
PGADMIN_DEFAULT_PASSWORD=admin123

# Backend
PORT=8080

DB_URL=jdbc:postgresql://db:5432/edufeed
DB_USER=edufeed
DB_PASSWORD=edufeed
SPRING_PROFILES_ACTIVE=dev

# Security (dev - NO usar en producción)
JWT_SECRET=dev-secret-key-not-for-production

```

4. Levantar base de datos

Usando VS Code tasks (recomendado):

1. Abrir carpeta en VS Code: ``code .``
2. ``Ctrl+Shift+P`` → ``Tasks: Run Task`` → ``DB: up``

Usando script PowerShell:

```

.\scripts\db-up.ps1

```

Usando Docker Compose directo:

```

docker compose up -d db pgadmin

```

Verificar:

```

docker ps

# Debe mostrar: edufeed-db (healthy), edufeed-pgadmin

```

5. Compilar proyecto

```

# Windows
$env:JAVA_HOME = "C:\Program Files\Java\jdk-24"
mvn clean install -DskipTests

# Linux
export JAVA_HOME=/opt/jdk-24
mvn clean install -DskipTests

```

6. Ejecutar backend

Opción A: VS Code task:

`Ctrl+Shift+P` → `Tasks: Run Task` → `Backend: run`

Opción B: Maven directo:

```
mvn -f edufeed-backend/pom.xml spring-boot:run
```

Verificar en <http://localhost:8080/actuator/health>

7. Ejecutar desktop app

EduFeed

6. MANUAL DE USUARIO

Manual de Usuario - Sistema EduFeed

Versión: 2.0

Fecha: 31 de octubre de 2025

Audiencia: Administradores, Cajeros y Operadores de Acceso

Índice

1. [Introducción](#introducción)
2. [Acceso al sistema](#acceso-al-sistema)
3. [Módulo de Administración](#módulo-de-administración)
4. [Módulo de Caja](#módulo-de-caja)
5. [Módulo de Acceso](#módulo-de-acceso)
6. [Reportes](#reportes)
7. [Preguntas frecuentes](#preguntas-frecuentes)
8. [Glosario](#glosario)

Introducción

¿Qué es EduFeed?

EduFeed es un sistema integral para la gestión del restaurante escolar que permite:

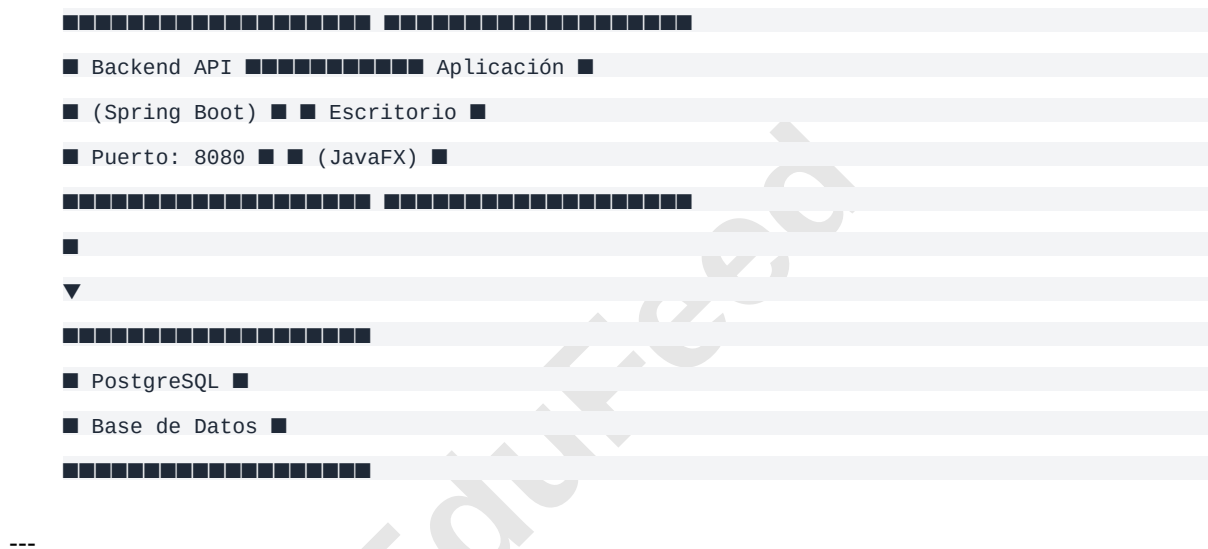
- ■ Registro y gestión de usuarios (estudiantes, docentes)
- ■ Validación biométrica de acceso (huella, rostro, voz)
- ■ Control de derechos adquiridos por pago (diario, mensual, paquetes)
- ■ Registro de transacciones en caja
- ■ Generación de reportes de asistencia y pagos

- ■ Auditoría completa de operaciones

Roles del sistema

Rol	Descripción	Módulos accesibles
Administrador	Gestiona usuarios, configura sistema, visualiza todos los reportes	Todos
Cajero	Registra pagos, emite recibos, consulta estado de cuenta	Caja, Reportes de caja
Operador de Acceso	Valida ingreso al restaurante mediante biometría	Acceso, Reportes de asistencia

Arquitectura del sistema



Accesso al sistema

1. Iniciar sesión

Aplicación de escritorio:

1. Abrir **EduFeed Desktop** desde el acceso directo
2. En la pantalla de inicio, ingresar:
 - **Usuario:** Tu nombre de usuario asignado (ej. `admin`, `cajero01`, `operador01`)
 - **Contraseña:** Tu contraseña personal
3. Hacer clic en "**Iniciar sesión**"
4. El sistema validará tus credenciales y mostrará el menú principal según tu rol

Notas de seguridad:

- ■■ No compartir tu contraseña con nadie

- ■■ Cerrar sesión al terminar tu jornada
- ■■ La sesión caduca automáticamente después de 30 minutos de inactividad

2. Menú principal

Después del login, verás el menú principal con opciones según tu rol:

Administrador ve:

- ■ Gestión de Usuarios
- ■ Gestión de Pagos
- ■ Validación de Acceso
- ■ Reportes

Cajero ve:

- ■ Registro de Pagos
- ■ Consultar Estado de Cuenta
- ■ Reportes de Caja

Operador de Acceso ve:

- ■ Validación de Acceso
- ■ Reportes de Asistencia

Módulo de Administración

Rol requerido: Administrador

3.1 Gestión de Usuarios

Registrar nuevo usuario

1. En el menú principal, hacer clic en "**Gestión de Usuarios**"
2. Hacer clic en el botón "**+ Nuevo Usuario**"
3. Completar el formulario:

Campo	Descripción	Ejemplo	Obligatorio
Tipo de Documento	CC, TI, CE, Pasaporte	CC	Sí
Número de Documento	Identificación única	1234567890	Sí
Nombres	Nombre(s) completo(s)	Juan Carlos	Sí
Apellidos	Apellido(s) completo(s)	Pérez Gómez	Sí

Campo	Descripción	Ejemplo	Obligatorio
Fecha de Nacimiento	Formato DD/MM/AAAA	15/03/2010	Sí
Tipo de Usuario	Niño, Estudiante, Docente	Estudiante	Sí
Grado	Si es estudiante	8°	No
Email	Correo electrónico	jperez@colegio.edu.co	No

4. Hacer clic en "**Capturar Biometría**"

Captura biométrica

El sistema soporta tres modalidades de biometría:

A. Captura de huella dactilar

1. Conectar el lector de huella USB
2. Seleccionar "**Huella**" en el selector de modalidad
3. Hacer clic en "**Iniciar Captura**"
4. Indicar al usuario que coloque el dedo índice en el lector
5. El sistema capturará automáticamente 3 muestras
6. Hacer clic en "**Confirmar**" cuando veas ■ "Captura exitosa"

B. Captura de rostro

1. Conectar la cámara web
2. Seleccionar "**Rostro**" en el selector de modalidad
3. Hacer clic en "**Iniciar Captura**"
4. La vista previa mostrará el rostro detectado con un recuadro verde
5. Ajustar parámetros si es necesario:
 - **Fuente:** Seleccionar cámara si hay varias
 - **Escala:** 1.1 (detección más sensible) a 1.5 (menos sensible)
 - **Vecinos mínimos:** 3-6 (filtro de falsos positivos)
6. Pedir al usuario que mire a la cámara con expresión neutral
7. Hacer clic en "**Capturar**" cuando el rostro esté centrado
8. El sistema guardará la imagen y los vectores faciales

C. Captura de voz

1. Conectar el micrófono
2. Seleccionar "**Voz**" en el selector de modalidad

3. Hacer clic en **"Iniciar Captura"**
4. La vista de forma de onda mostrará el audio en tiempo real
5. Pedir al usuario que lea la frase mostrada en pantalla:
 - Ejemplo: *"Mi nombre es Juan Carlos Pérez"*
6. Hacer clic en **"Detener"** después de 3-5 segundos
7. Reproducir el audio capturado para verificar calidad
8. Hacer clic en **"Confirmar"** si el audio es claro

Recomendaciones:

- ■ Capturar al menos 2 modalidades (ej. huella + rostro)
 - ■ Asegurar buena iluminación para captura de rostro
 - ■ Usar micrófono de calidad para voz
 - ■■ No forzar el dedo en el lector (puede causar rechazo)
5. Revisar datos ingresados
 6. Hacer clic en **"Guardar Usuario"**
 7. El sistema asigna un ID único y muestra confirmación

Editar usuario existente

1. En **"Gestión de Usuarios"**, buscar por nombre o documento
2. Hacer doble clic en el usuario en la tabla
3. Modificar campos necesarios
4. Si necesitas recapturar biometría:
 - Hacer clic en **"Recapturar Biometría"**
 - Seleccionar modalidad (huella/rostro/voz)
 - Seguir proceso de captura
5. Hacer clic en **"Actualizar"**

Eliminar usuario

1. Buscar el usuario en la lista
2. Hacer clic derecho → **"Eliminar"**
3. Confirmar la acción (■■ No reversible)

Nota: Solo usuarios sin pagos pendientes o accesos recientes pueden eliminarse.

3.2 Gestión de Operadores

Los operadores son usuarios del sistema (administradores, cajeros, operadores de acceso).

Crear operador

1. Ir a "**Gestión de Operadores**"
2. Hacer clic en "**+ Nuevo Operador**"
3. Completar formulario:
 - **Nombre de Usuario:** `operador01` (único, sin espacios)
 - **Contraseña:** Mínimo 8 caracteres, una mayúscula, un número
 - **Rol:** Administrador, Cajero o Operador de Acceso
 - **Nombre Completo:** Nombre y apellido
 - **Email:** Para notificaciones
4. Hacer clic en "**Crear**"

Cambiar contraseña de operador

1. En la lista de operadores, seleccionar el usuario
2. Hacer clic en "**Cambiar Contraseña**"

EduFeed

7. REGISTRO INDIVIDUAL DE MODALIDADES

■ Registro Individual de Modalidades Biométricas

■ Resúmen

Flujo para registrar **una sola modalidad** biométrica (huella, rostro o voz) sin forzar el flujo completo de 3 pasos. Ideal para:

- Actualizar una modalidad específica sin re-registrar todas
- Agregar modalidad faltante (ej: usuario solo tiene huella, quiere agregar rostro)
- Reemplazar modalidad inválida (ej: rostro mal capturado, volver a registrar)

■ Problema Resuelto

Antes (V2):

Admin selecciona "Rostro" → QR muestra wizard 3 pasos

Usuario escanea QR → Obligado a registrar huella + rostro + voz

Usuario solo quería actualizar rostro → Frustración

Ahora (V3):

Admin selecciona "Rostro" → QR específico de rostro

Usuario escanea QR → Solo aparece tarjeta de rostro

Usuario registra rostro → Página cierra automáticamente

■ ■ Arquitectura

■ Desktop (JavaFX)■

■ UserManagement ■

[illegible]

```
■ showSingleModalityQR(modalidad)
```

[illegible][illegible]

■ BiometricRegistrationController■

Auto-creación de sesión en páginas individuales:

```
@GetMapping("/fingerprint")
public String fingerprintRegistrationPage(
    @RequestParam(value = "sessionId", required = false) String sessionId,
    @RequestParam(value = "userId", required = false) String userId,
    Model model
) {
    // Si no hay sesión válida, crear una nueva
    if (sessionId == null || registrationService.getUserFromSession(sessionId) == null) {
        Usuario user = usuarioRepository.findByDocumento(userId).orElse(null);
        if (user != null) {
            sessionId = registrationService.startSession(user);
        }
    }
    model.addAttribute("sessionId", sessionId);
    return "biometric-register-fingerprint";
}
// Mismo patrón para /face y /voice
```

2. Frontend

Vista Principal

`biometric-register.html`

```
<script th:inline="javascript">
const onlyType = /*[[${onlyType}]]*/ ''; // "huella" | "rostro" | "voz" | ''
function applyOnlyTypeMode() {
    if (!onlyType) return; // Modo normal (3 modalidades)
    console.log('[ONLY-TYPE] Modo individual activado:', onlyType);
    // Ocultar tarjetas no seleccionadas
    const cards = {
        'huella': 'cardFingerprint',
        'rostro': 'cardFace',
        'voz': 'cardVoice'
    };
    Object.entries(cards).forEach(([type, cardId]) => {
        const card = document.getElementById(cardId);
        if (type !== onlyType && card) {
            card.style.display = 'none';
        }
    });
}
```


3. Desktop

UserManagementModule

`UserManagementModule.java`

```
// Botones individuales por modalidad
private void doRegistrarModalidad(UserApiClient.Modalidad modalidad) {
    String userId = usuarioSeleccionado.getDocumento();
    String token = UUID.randomUUID().toString();
    String sessionId = UUID.randomUUID().toString();
    // Mapear modalidad a parámetro 'type'
    String typeParam = modalidad.name().toLowerCase();
    if (modalidad == UserApiClient.Modalidad.ROSTRO) typeParam = "rostro";
    else if (modalidad == UserApiClient.Modalidad.VOZ) typeParam = "voz";
    else typeParam = "huella";
    // URL con parámetro 'type'
    String url = String.format(
        "%s/api/biometric/register?userId=%s&token=%s&sessionId=%s&type=%s",
        baseUrl, userId, token, sessionId, typeParam
    );
    showSingleModalityQR(
        usuarioSeleccionado.getNombreCompleto(),
```

■ Cómo Probar

1. Registro Individual de Rostro

```
# 1. Levantar servicios
.\scripts\db-up.ps1

mvn -f edufeed-backend\pom.xml spring-boot:run

# 2. Desktop
mvn -f edufeed-desktop\pom.xml javafx:run

# 3. Admin → Usuarios → Seleccionar usuario → Botón "Rostro"
# 4. Escanear QR → Solo aparece tarjeta de rostro
# 5. Registrar rostro → Auto-cierre tras mensaje "■ ¡Registro completado!"
```

2. Verificar en Base de Datos

```
SELECT
u.nombre_completo,
pb.modalidad,
pb.activo,
```

```

pb.created_at
FROM plantilla_biometrica pb
JOIN app_user u ON pb.usuario_id = u.id
WHERE u.documento = '123456'
ORDER BY pb.created_at DESC;
-- Debe mostrar solo 1 registro nuevo con modalidad = 'ROSTRO'

```

3. URL Manual (opcional)

```

http://localhost:8080/api/biometric/register?userId=123456&token=abc123&sessionId=xyz789&type=rostro

```

■ Debugging

Logs Backend

```

[BIO-REG] Modo individual activado: type=rostro
[BIO-REG] Usuario 123456 tiene sesión: xyz789
[BIO-REG] Sesión auto-creada para userId=123456
[BIO-REG] Rostro registrado exitosamente para user 123456

```

Logs Frontend (Console)

```

[ONLY-TYPE] Modo individual activado: rostro
[ONLY-TYPE] Ocultando tarjetas: huella, voz
[ONLY-TYPE] Auto-abriendo modalidad: rostro
[REGISTER] Rostro capturado, enviando...
[UI] 1 de 1 completados
[UI] Cerrando página en 1.5s

```

Logs Desktop (Console)

```

[QR-DIALOG] Modalidad: ROSTRO, URL generada
[QR-DIALOG] Polling iniciado para user=123456, session=xyz789
[QR-DIALOG] Verificando status... rostroCompletado=false
[QR-DIALOG] Verificando status... rostroCompletado=true
[QR-DIALOG] ■ Registro completado, cerrando diálogo

```

■ Casos de Error

Error: "Sesión no encontrada o expirada"

Causa: SessionId inválido o expirado

Solución: Backend auto-crea sesión si userId está presente

```
if (sessionId == null || registrationService.getUserFromSession(sessionId) == null) {  
    sessionId = registrationService.startSession(user);  
}
```

Error: Página no cierra automáticamente

Causa: `window.close()` bloqueado por navegador (no abierta via script)

Solución: Usuario cierra manualmente tras ver mensaje "■ ¡Registro completado!"

Error: QR no actualiza status

Causa: Polling no detecta cambio en BD

Verificar:

```
SELECT rostro_completado FROM biometric_registration_session  
WHERE session_id = 'xyz789';
```

EduFeed

8. PATRONES UX - PREVIEW Y GRABACION

■ Patrones UX: Preview y Grabación en Autenticación Biométrica

■ Resúmen

Guía de patrones de experiencia de usuario para captura biométrica en EduFeed V3. Incluye:

- **Preview en vivo** para FaceID con detección facial en tiempo real
- **Grabación explícita** para voz con timer y controles visuales
- **Feedback inmediato** para guiar al usuario durante la captura

■ Problema Resuelto

Antes (V2)

FaceID: Captura instantánea sin preview → Usuario no sabe si está encuadrado

Voz: Acepta cualquier ruido inmediatamente → Grabaciones de 0.1s inválidas

Sin feedback visual → Usuario no sabe si la captura fue exitosa

Ahora (V3)

FaceID: Preview con recuadros verdes sobre rostros detectados

Usuario ve si está bien encuadrado antes de capturar

Botón explícito "Capturar y Verificar"

Voz: Botón micrófono para iniciar/detener grabación

Timer visible (mínimo 3 segundos)

Waveform animado mientras graba

■ ■ Arquitectura

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., publishing as Pearson Benjamin Cummings, 101 Philip Drive, Assinippi Park, New York, NY 10964-2133.

■ biometric-auth.html ■

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., publishing as Pearson Benjamin Cummings, 101 Philip Drive, Assinippi Park, New York, NY 10984.

■ ■ FaceID Preview ■ ■

■ ■ ■ ■■ Status: Grabando ■ ■ ■

```

.face-preview-container {
position: relative;
width: 100%;
max-width: 480px;
margin: 20px auto;
border-radius: 12px;
overflow: hidden;
box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
#faceVideo {
width: 100%;
height: auto;
display: block;
background: #000;
}
#faceCanvas {
position: absolute;
top: 0;

```

JavaScript Logic

```

let faceStream = null;
let faceDetectionInterval = null;
async function authenticateFace() {
try {
// Cargar modelos de face-api.js (solo una vez)
await ensureFaceApiModels();
// Solicitar cámara
faceStream = await navigator.mediaDevices.getUserMedia({
video: { facingMode: 'user', width: 480, height: 640 }
});
// Asignar stream a video
const video = document.getElementById('faceVideo');
video.srcObject = faceStream;
await video.play();
// Mostrar preview
document.getElementById('facePreviewContainer').style.display = 'block';

```

Gestión de Modelos

```

let faceApiLoaded = false;
async function ensureFaceApiModels() {
if (faceApiLoaded) return;

```

```

console.log('[FACE-API] Cargando modelos...');
const modelPath = '/webjars/face-api/models';
await Promise.all([
  faceapi.nets.ssdMobilenetv1.loadFromUri(modelPath),
  faceapi.nets.faceLandmark68Net.loadFromUri(modelPath),
  faceapi.nets.faceRecognitionNet.loadFromUri(modelPath)
]);
faceApiLoaded = true;
console.log('[FACE-API] Modelos cargados exitosamente');
}

```

2. Grabación de Voz

HTML Structure

```

<!-- Contenedor de grabación -->
<div id="voiceRecorder" class="voice-recorder" style="display:none;">
  <div class="recorder-status">
    <div class="waveform" id="voiceWaveform">
      <div class="wave-bar"></div>
      <div class="wave-bar"></div>
      <div class="wave-bar"></div>
      <div class="wave-bar"></div>
      <div class="wave-bar"></div>
    </div>
    <div class="timer" id="voiceTimer">00:00</div>
    <div class="status-text" id="voiceStatusText">Listo para grabar</div>
  </div>
  <div class="recorder-controls">
    <button id="btnMic" class="mic-btn">
      ■ Iniciar Grabación
    </button>
  </div>
</div>

```

CSS Styles

```

.voice-recorder {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border-radius: 16px;
  padding: 30px;
  color: white;
  text-align: center;
}

```

```

}
.recorder-status {
margin-bottom: 30px;
}
.waveform {
display: flex;
justify-content: center;
align-items: center;
gap: 4px;
height: 60px;
margin-bottom: 15px;
}

```

JavaScript Logic

```

let voiceRecorder = null;
let voiceChunks = [];
let voiceRecording = false;
let voiceSeconds = 0;
let voiceTimer = null;
async function toggleVoiceRecording() {
if (!voiceRecording) {
await startVoiceRecording();
} else {
stopVoiceRecording();
}
}
async function startVoiceRecording() {
try {
// Solicitar micrófono
const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
// Crear MediaRecorder

```


9. ESTRUCTURA DEL REPOSITORIO

Estructura del repositorio — EduFeed

Este documento describe qué hace cada carpeta y archivo importante del proyecto, cómo están organizados los módulos, y pasos rápidos para ejecutar y desarrollar localmente.

> Fecha: 2025-10-10

Resumen de alto nivel

EduFeed es un proyecto multi-módulo Maven que contiene:

- ``edufeed-backend``: API REST (Spring Boot) y lógica del dominio.
- ``edufeed-desktop``: aplicación JavaFX para punto de control y caja.
- ``edufeed-biometric``: librería/abstracción para proveedores biométricos (mock + plugins).
- ``edufeed-common``: modelos/DTOs compartidos.
- Infraestructura: ``docker-compose.yml``, scripts de arranque (``scripts/``), y documentación (``docs/``).

Carpetas y archivos raíz

- ``pom.xml`` (root)
- POM padre del multi-módulo. Define versiones comunes (Java, Spring Boot), `pluginManagement` y módulos.
- ``docker-compose.yml``
- Define servicios Docker para desarrollo: ``db`` (Postgres) y ``pgadmin``.
- Variables leídas desde ``env``.
- ``env`` (no subir credenciales sensibles)
- Variables de entorno para facilitar el desarrollo local (DB, pgAdmin, backend port, etc.).
- ``README.md``
- Documentación general del reto, requerimientos funcionales y NO funcionales, y comandos resumen.

- ``docs/`` (documentación)
- ``Contexto.md`` — bitácora y decisiones de arquitectura.
- ``REPO_STRUCTURE.md`` — este archivo.
- ``architecture.md``, ``manual-instalacion.md``, ``manual-usuario.md`` — otros documentos de soporte.
- ``scripts/``
 - ``db-up.ps1`` / ``db-down.ps1`` — scripts PowerShell para levantar y bajar la DB y pgAdmin, con esperas/healthchecks.
 - ``setup-dev.ps1`` / ``setup-dev.sh`` — scripts de instalación de dependencias locales y configuración del entorno.
 - ``reportes/`` — SQL y scripts para generar/reportes.
- ``seed/``
 - SQL de seed con datos de prueba.

Módulos Maven

edufeed-backend/

Estructura principal del backend:

- ``pom.xml`` — configuración específica del módulo (dependencias Spring Boot, Flyway, springdoc, etc.).
- ``src/main/java/co/cellano/edufeed/backend/`` — código Java:
- ``EduFeedApplication.java`` — clase ``main`` de Spring Boot.
- ``config/`` — clases de configuración (DataSource, seguridad, TimeZone, Flyway overrides).
- ``controller/`` — controladores REST (por ejemplo ``HealthController``, ``BiometricTestController``, endpoints de prueba).
- ``service/`` — servicios de negocio (lógica central: acceso, pagos, etc.).
- ``repository/`` — interfaces Spring Data JPA (si existen).
- ``model/`` — entidades JPA y DTOs si aplica.
- ``db/migration/`` — scripts Flyway (V1__init.sql y futuros V2, V3...).
- ``src/main/resources/``:
 - ``application.yml`` — configuración de la app (datasource, JPA, Flyway, logging, springdoc, actuator).
 - ``application.properties`` — propiedades planas (usadas para evitar advertencias de lint en YAML), p. ej. ``spring.jpa.properties.hibernate.jdbc.time_zone``.
 - ``META-INF/spring-configuration-metadata.json`` — metadata que ayuda al IDE con propiedades custom.

Notas:

- Flyway se usa para migraciones. Si añades scripts, ponlos en ``src/main/resources/db/migration`` con la convención ``V{n}__descripcion.sql``.

- Las tablas y nombres actuales (según V1) están en español: `usuarios`, `accesos`, `pagos`, `derechos\_uso`, etc.

edufeed-desktop/

- `pom.xml` — dependencias JavaFX y plugin `javafx-maven-plugin`.
- `src/main/java/...` — código JavaFX, clases de UI, servicios que consumen la API del backend.
- Propósito: app de escritorio para realizar lecturas biométricas, visualización de estado de acceso y caja.

edufeed-biometric/

- Interfaces para `BiometricProvider` y una implementación `MockBiometricProvider`.
- Aquí integrarán SDKs/hardwares reales en el futuro.

edufeed-common/

- DTOs, enums y utilidades compartidas entre backend y desktop.

Archivos de interés y cómo usarlos

- `edufeed-backend/src/main/resources/db/migration/V1\_\_init.sql`
- Contiene la estructura base de la BD: tablas `usuarios`, `plantillasbiometricas`, `pagos`, `paquetespago`, `derechosuso`, `accesos`, `usospaquete`, `auditoria`, `transaccionescaja`, `calendarioservicio`.
- Al iniciar el backend Flyway aplica estas migraciones si aún no existen.
- `edufeed-backend/src/main/resources/application.yml` y `application.properties`
- Contienen las variables de configuración. En local usamos `.env` para inyectar valores en `docker-compose` y la aplicación (cuando se ejecuta desde Maven se leen del entorno del proceso).
- `.vscode/tasks.json`
- Tareas preconfiguradas para arrancar backend y desktop desde VS Code (usa `mvn -f edufeed-backend/pom.xml spring-boot:run`).
- `docker-compose.yml`
- Define `db` y `pgadmin`. Usa volúmenes `dbdata` y `pgadmindata`.

Cómo empezar a desarrollar (rápido)

1. Asegúrate de tener Java 21 (o compatible configurado en VS Code), Maven y Docker Desktop.
2. Copia `.env.example` a `.env` y ajusta credenciales si hace falta.

3. Levanta la BD y pgAdmin:

```
./scripts/db-up.ps1
```

4. Ejecuta el backend (desde la raíz):

```
mvn -f edufeed-backend/pom.xml spring-boot:run
```

5. Ejecuta el desktop (si necesitas UI):

```
mvn -f edufeed-desktop/pom.xml -DskipTests javafx:run
```

6. Abre pgAdmin en `http://localhost:5050` y registra el servidor ``db`` con credenciales de ``.env``.

Convenciones y notas de diseño

- Las migraciones Flyway son la fuente de la verdad para el esquema. Evita cambios manuales en la BD en producción: siempre crea una nueva migración.
- Nombres de tablas actuales están en español por claridad con stakeholders. Podemos crear vistas en inglés para compatibilidad si migras código o herramientas externas.
- Biometría: la librería ``edufeed-biometric`` expone una interfaz; usa ``MockBiometricProvider`` para pruebas end-to-end.

Próximos cambios / recomendaciones inmediatas

1. Implementar el vertical slice de ``access/check`` + ``payments`` (migraciones V2) para controlar acceso según derecho adquirido.
2. Agregar tests unitarios e integración (contenerizados) y CI.
3. Preparar scripts de seed y vistas de reportes automáticos.
4. Documentar API con ejemplos curl en README.

Si quieres, puedo:

- Añadir un índice al README con enlaces a cada documento.
- Generar ``docs/DEVELOPER_SETUP.md`` con pasos detallados para configurar IDE, maven/Java y debug en VS Code.

Marca la opción que prefieras y lo incorporo.

10. TROUBLESHOOTING

Guía de Troubleshooting - Sistema EduFeed

Versión: 2.0

Fecha: 31 de octubre de 2025

Audiencia: Administradores de sistemas, DevOps, Desarrolladores

Índice

1. [Introducción](#introducción)
2. [Problemas de compilación](#problemas-de-compilación)
3. [Problemas de Docker](#problemas-de-docker)
4. [Problemas de base de datos](#problemas-de-base-de-datos)
5. [Problemas de despliegue CI/CD](#problemas-de-despliegue-cicd)
6. [Problemas de observabilidad](#problemas-de-observabilidad)
7. [Problemas de backups](#problemas-de-backups)
8. [Problemas de la aplicación](#problemas-de-la-aplicación)
9. [Problemas de hardware biométrico](#problemas-de-hardware-biométrico)
10. [Diagnóstico general](#diagnóstico-general)
11. [Checklist de verificación](#checklist-de-verificación)

Introducción

Esta guía consolida todos los problemas comunes documentados en las fases de desarrollo del sistema EduFeed. Cada problema está estructurado con:

- **Síntoma:** Qué se observa
- **Causa:** Por qué ocurre

- **Diagnóstico:** Cómo investigar
- **Solución:** Cómo resolverlo
- **Prevención:** Cómo evitarlo en el futuro

Formato de comandos

```
# Linux/Mac
comando_linux

# Windows PowerShell
comando_powershell
```

Problemas de compilación

Maven: "Cannot resolve dependency"

Síntoma: Build falla con error de dependencia no encontrada.

Causa: Módulos `edufeed-common` o `edufeed-biometric` no compilados antes de `edufeed-backend` o `edufeed-desktop`.

Diagnóstico:

```
# Verificar repositorio local Maven
ls ~/.m2/repository/co/cellano/

# Debe contener edufeed-common, edufeed-biometric
```

Solución:

```
# Compilar en orden correcto
mvn clean install -DskipTests

# O compilar módulos comunes primero
cd edufeed-common
mvn clean install
cd ../edufeed-biometric
mvn clean install
cd ../edufeed-backend
mvn clean package
```

Prevención: Usar Maven multi-module build con reactor order correcto (ya configurado en pom.xml raíz).

Error: "Class not found: java.lang.UnsupportedClassVersionError"

Síntoma: Compilación o ejecución falla con error de versión de clase.

Causa: JDK/JRE versión incorrecta (requiere JDK 24).

Diagnóstico:

```
# Verificar versión Java
java -version
javac -version
# Verificar JAVA_HOME
echo $env:JAVA_HOME # Windows
echo $JAVA_HOME # Linux
```

Solución:

```
# Windows
$env:JAVA_HOME = "C:\Program Files\Java\jdk-24"
$env:PATH = "$env:JAVA_HOME\bin;$env:PATH"
# Linux
export JAVA_HOME=/opt/jdk-24
export PATH=$JAVA_HOME/bin:$PATH
# Verificar
java -version
# openjdk version "24" ...
```

Prevención: Documentar versión en README y validar en scripts de build.

Maven: "Tests failed"

Síntoma: `mvn test` falla pero tests pasan en IDE.

Causa: Diferencias de entorno (zona horaria, base de datos, configuración).

Diagnóstico:

```
# Ver detalles de falla
mvn test -X # modo debug
# Ver reporte Surefire
cat edufeed-backend/target/surefire-reports/*.txt
```

Solución:

```
# Asegurar Testcontainers funciona
docker ps

# Debe mostrar contenedores de test (postgres)

# Configurar zona horaria

# En pom.xml o application-test.yml
spring:
  jpa:
  properties:
  hibernate:
  jdbc:
  time_zone: UTC

# Ejecutar con profile de test
mvn test -Dspring.profiles.active=test
```

Prevención: Usar Testcontainers para tests de integración (ya implementado).

Problemas de Docker

Error: "Cannot connect to Docker daemon"

Síntoma: `docker ps` falla con error de conexión.

Causa: Docker Desktop no está corriendo o permisos insuficientes.

Diagnóstico:

```
# Verificar estado Docker
docker info

# Si falla: Docker no está corriendo

# Windows: verificar Docker Desktop
Get-Process | Where-Object { $_.Name -like "*Docker*" }

# Linux: verificar servicio
sudo systemctl status docker
```

Solución:

```
# Windows: iniciar Docker Desktop manualmente

# O desde PowerShell
```



```
Start-Process "C:\Program Files\Docker\Docker\Docker Desktop.exe"
# Linux: iniciar servicio
sudo systemctl start docker
# Agregar usuario a grupo docker (Linux)
sudo usermod -aG docker $USER
newgrp docker # o logout/login
```

Prevención: Configurar Docker para iniciar al boot del sistema.

Error: "port is already allocated"

Síntoma: `docker compose up` falla con error de puerto ocupado.

Causa: Puerto (5432, 8080, 5050, etc.) ocupado por otra aplicación.

Diagnóstico:

```
# Windows: ver qué proceso usa el puerto
netstat -ano | findstr :8080
# Última columna es PID
tasklist | findstr <PID>
# Linux
sudo lsof -i :8080
# Muestra proceso
```

Solución:

```
# Opción 1: Detener proceso conflictivo
# Windows
taskkill /PID <PID> /F
# Linux
sudo kill -9 <PID>
# Opción 2: Cambiar puerto en .env
PORT=8081
# Opción 3: Detener contenedores anteriores
docker compose down
```

Prevención: Usar puertos no estándar (ej. 8081 para staging) o detener servicios antes de levantar.

11. CHANGELOG V3 - NOVIEMBRE 2025

V3 — Registro de cambios (Noviembre 2025)

■ Sistema de Autenticación Biométrica Avanzado

Fecha: 10-14 de noviembre de 2025

Este documento detalla TODAS las mejoras implementadas en el sistema de autenticación biométrica, incluyendo WebAuthn, reconocimiento facial con IA, verificación de voz, y flujos de registro individuales.

■ Resumen Ejecutivo

Se implementó un **sistema biométrico completo de nivel empresarial** con:

- ■ **WebAuthn real** con passkeys para huella digital (no simulación)
- ■ **Reconocimiento facial ML** con face-api.js (descriptores 128D + matching)
- ■ **Verificación de voz** con MFCC y similitud coseno
- ■ **Registro individual** de modalidades sin forzar las 3
- ■ **Preview en vivo** para rostro y grabación explícita para voz
- ■ **Desktop reactivo** con notificación instantánea tras auth exitosa

■ Funcionalidades Implementadas

1. WebAuthn con Passkeys Reales

Backend

``WebAuthnController.java`` [NUEVO]

- ``POST /api/webauthn/registro/iniciar`` - Genera challenge del servidor
- ``POST /api/webauthn/registro/{sesionId}/completar`` - Completa registro y guarda public key

- `POST /api/webauthn/autenticacion/iniciar` - Inicia flujo de login con challenge
- `POST /api/webauthn/autenticacion/{sesionId}/completar` - Verifica firma completa

`WebAuthnService.java` [NUEVO]

```
@Service
public class WebAuthnService {
    // Registro
    Map<String, Object> iniciarRegistro(String usuarioDocumento);
    Map<String, Object> completarRegistro(String sesionId, WebAuthnRegistroRequest req);
    // Autenticación
    Map<String, Object> iniciarAutenticacion(String usuarioDocumento);
    Map<String, Object> completarAutenticacion(String sesionId, WebAuthnAutenticacionRequest req);
    // Verificación completa
    - Parseo CBOR de authenticatorData
    - Extracción de credentialId, flags, signCount
    - Validación de signature sobre authenticatorData || SHA-256(clientDataJSON)
    - Soporte EC (P-256) y RSA (RS256)
    - Actualización de signCount anti-replay
}
```

Modelos

- `WebAuthnCredencial.java` - Entity JPA para credenciales
- `WebAuthnSesion.java` - Sesiones de registro/autenticación
- `WebAuthnRegistroRequest/Response.java`
- `WebAuthnAutenticacionRequest/Response.java`
- Repositorios: `WebAuthnCredencialRepository`, `WebAuthnSesionRepository`

Frontend

`biometric-register-fingerprint.html` [MODIFICADO]

- Flujo server-driven: llama `api/webauthn/registro/iniciar` para challenge
- Crea credential platform con `residentKey: "preferred"``
- Extrae public key PEM con función `getPublicKey()``
- Completa registro backend y notifica wizard legacy

`static/pwa-webauthn.html` [MODIFICADO]

- Incluye PEM public key en payload cuando navegador lo soporta
- Mejorada UX de QR y feedback

`biometric-auth.html` [MODIFICADO]

- Login WebAuthn completo:

1. Obtiene challenge y allowCredentials

2. Llama ``navigator.credentials.get()``
3. Envía signature para verificación backend
 - Normalización credentialId (base64/base64url compatible)
 - Desktop notificado inmediatamente tras auth exitosa

2. Reconocimiento Facial con IA

Modelos ML

- `face-api.js` (vladmandic fork) cargado desde CDN
- ``ssdMobilenetv1`` - Detección de rostros
- ``faceLandmark68Net`` - Landmarks faciales
- ``faceRecognitionNet`` - Descriptores 128D

Registro

``biometric-register-face.html`` [MODIFICADO]

```
// Captura con BlazeFace para preview
// Genera descriptor 128D con face-api.js
const detection = await faceapi.detectSingleFace(img)
  .withFaceLandmarks()
  .withFaceDescriptor();
const descriptor = Array.from(detection.descriptor); // [128 floats]
// Envía JSON: {descriptor: [...]}
await fetch('/api/biometric/register/face', {
  body: JSON.stringify({ sessionId, faceData: JSON.stringify({descriptor}) })
});
```

Acceso

``biometric-auth.html`` [MODIFICADO - Preview en vivo]

```
// Preview de cámara con detección en tiempo real
async function authenticateFace() {
  await ensureFaceApiModels();
  faceStream = await navigator.getUserMedia({video: true});
  faceVideo.srcObject = faceStream;
  startFaceDetectionLoop(); // Dibuja recuadros verdes sobre rostros
}

// Captura explícita con botón
async function captureFace() {
  const detection = await faceapi.detectSingleFace(canvas)
    .withFaceLandmarks()
    .withFaceDescriptor();
```

```

if (detection) {
  const payload = JSON.stringify({descriptor: Array.from(detection.descriptor)});
  // Envía para verificación
}
}

```

Estilos agregados

```

.face-preview-container {
  position: relative;
  /* Video + canvas overlay para detección */
}
#faceCanvas {
  position: absolute;
  top: 0; left: 0;
  /* Dibuja recuadros sobre video */
}

```

Backend

`BiometricAuthService.java` [MODIFICADO]

```

public boolean verifyFaceId(Usuario user, String faceData) {
  if (faceData.trim().startsWith("{")) {
    // Parse JSON descriptor
    var node = mapper.readTree(faceData);
    double[] probe = toDoubleArray(node.get("descriptor"));
    // Obtener template guardado
    var plantillas = plantillaBiometricaRepository
      .findByUsuarioIdAndActivoTrue(user.getId())
      .stream().filter(p -> p.getModalidad() == Modalidad.ROSTRO).toList();
    for (var p : plantillas) {
      if (p.getPlantilla()[0] == '{') { // JSON descriptor
        var stored = mapper.readTree(new String(p.getPlantilla(), UTF_8));
        double[] ref = toDoubleArray(stored.get("descriptor"));
        double dist = l2(probe, ref); // Distancia euclidiana
        if (dist < 0.6) return true; // Umbral típico
      }
    }
  }
}

```

`BiometricRegistrationService.java` [MODIFICADO]

- Acepta JSON descriptors (UTF-8) además de base64
- Permite ML pipelines futuros

3. Verificación de Voz con MFCC

Registro

`biometric-register-voice.html` [MODIFICADO]

```
// Grabación con MediaRecorder + MFCC
const blob = new Blob(voiceChunks, {type: 'audio/webm'});
const decoded = await offline.decodeAudioData(await blob.arrayBuffer());
const channel = decoded.getChannelData(0);
// Extrae MFCC promedio con Meyda
let mfccSum = null, count = 0;
for (let i = 0; i + frameSize <= channel.length; i += hop) {
  const feats = Meyda.extract('mfcc', frame, {
    sampleRate, bufferSize: 1024,
    melBands: 26, numberOfMFCCCoefficients: 13
  });
  if (feats.mfcc) {
    if (!mfccSum) mfccSum = new Array(13).fill(0);
    for (let k=0; k<13; k++) mfccSum[k] += feats.mfcc[k];
    count++;
  }
}
const avg = mfccSum.map(v => v/count);
// Envía JSON: {features: [13 floats]}
```

Acceso

`biometric-auth.html` [MODIFICADO - Grabación explícita]

```
// Botón micrófono con timer
let voiceRecording = false;
let voiceSeconds = 0;
async function toggleVoiceRecording() {
  if (!voiceRecording) {
    await startVoiceRecording(); // Inicia MediaRecorder + timer
  } else {
    stopVoiceRecording(); // Min 3s, procesa MFCC y envía
  }
}
function stopVoiceRecording() {
  if (voiceSeconds < 3) {
    showStatus('error', 'Grabación muy corta, mínimo 3 segundos');
    return;
  }
}
```

```
}  
// Extrae MFCC y verifica  
}
```

Estilos agregados

```
.voice-recorder.active {  
display: block; /* Muestra UI de grabación */
```

EduFeed

12. REPORTES ECONOMICOS

■ Remodelación Completa - Módulo de Reportes Económicos

■ Resumen Ejecutivo

Estado: ■ **COMPLETADO** (9/10 tareas - 90%)

Última Actualización: 11 de noviembre de 2025

Compilación: ■ **EXITOSA** sin errores

Se ha completado exitosamente la remodelación completa del módulo de Reportes de EduFeed, transformándolo de un sistema básico de reportes a un **Dashboard Financiero Premium** con análisis exhaustivo, visualizaciones avanzadas y capacidades de exportación profesional.

■ Objetivos Alcanzados

■ 1. Dashboard Financiero Completo

- **8 Tarjetas de Métricas:** Ingresos Totales, Devoluciones, Ingresos Netos, Rentabilidad %, Total Transacciones, Aprobadas, Revertidas, Ticket Promedio
- **3 Gráficas Interactivas:** LineChart (evolución temporal), BarChart (comparativa), PieChart (distribución por tipo)
- **Tabla Detallada:** 8 columnas con todas las transacciones, estados coloreados
- **Filtros Avanzados:** DatePickers personalizables + 4 botones rápidos (Hoy, Semana, Mes, Año)

■ 2. Exportación Profesional

- **PDF Premium con iText 7:**
- Portada corporativa con título "EduFeed" y período
- Resumen Ejecutivo (tabla 2x4 con métricas clave)
- Análisis Gráfico (LineChart y PieChart embebidos como PNG)
- Tabla completa de transacciones (8 columnas, estados coloreados)
- Marca de agua "EduFeed" semi-transparente en todas las páginas
- Footer con copyright "© 2025 EduFeed" y numeración

- **CSV Compatible Excel/DB:**

- UTF-8 con BOM para auto-detección en Excel
- Delimitador punto y coma (;) estándar español
- Headers en español: Fecha, Referencia, Usuario, Documento, Tipo Pago, Método Pago, Monto, Estado, Motivo Pago, Motivo Devolución
- Escape RFC 4180 (comillas duplicadas)
- Sección de resumen ejecutivo con métricas

■ 3. Arquitectura de Tabs Profesional

- **Tab 1 - ■ Económicos:** Dashboard financiero completo (NUEVO)
- **Tab 2 - ■ Asistencias:** Reporte de asistencia (legacy mantenido)
- **Tab 3 - ■ Rechazos:** Reporte de rechazos (legacy mantenido)
- **Tab 4 - ■ Derechos Activos:** Reporte de derechos (legacy mantenido)
- CSS personalizado con colores diferenciados por tab
- Navegación fluida con efectos hover y focus accessibility

■ 4. Cálculos Financieros Avanzados

```
// Rentabilidad
((ingresosTotales - devolucionesTotales) / ingresosTotales) * 100

// Ticket Promedio
ingresosTotales / cantidadTransaccionesAprobadas

// Ingresos Diarios
Agrupación por LocalDate con llenado de gaps (fechas sin datos = $0)

// Distribución por Tipo
Agrupación por TipoPago con cálculo de porcentajes relativos
```

■■ Arquitectura del Sistema

```
■ edufeed-desktop/src/main/java/co/cellano/edufeed/desktop/reports/
■■■ ■ models/
■ ■■■ FinancialReportData.java ■ DTO principal con nested classes
■ ■■■ TransactionSummary.java ■ DTO de transacción individual
■
■■■ ■ services/
■ ■■■ FinancialReportService.java ■ Lógica de negocio y cálculos
■ ■■■ PDFExportService.java ■ Generación PDF con iText 7
■ ■■■ CSVExportService.java ■ Generación CSV compatible Excel
```

```

■
■■■■ ■ views/
■ ■■■ FinancialDashboardView.java ■ UI premium con 8 cards + 3 charts
■
■■■■ ■ controllers/
■ ■■■ FinancialReportsController.java ■ Controlador con async/await
■
■■■■ ReportsView.java ■ Vista principal con TabPane
■■■■ ReportsModule.java ■ Módulo embebible actualizado
■■■■ ReportsController.java ■ Controlador legacy actualizado

```

■ Tecnologías y Dependencias

Nuevas Dependencias Agregadas

1. iText 7 Core 8.0.5 (PDF)

```

<dependency>
<groupId>com.itextpdf</groupId>
<artifactId>itext7-core</artifactId>
<version>8.0.5</version>
<type>pom</type>
</dependency>

```

Uso: Generación de PDFs profesionales con marca de agua, tablas coloreadas, gráficas embebidas

2. Apache POI 5.3.0 (Excel)

```

<dependency>
<groupId>org.apache.poi</groupId>
<artifactId>poi-ooxml</artifactId>
<version>5.3.0</version>
</dependency>

```

Uso: Reservado para futuras mejoras (actualmente usando CSV)

3. JFreeChart 1.5.5 (Gráficas)

```

<dependency>
<groupId>org.jfree</groupId>
<artifactId>jfreechart</artifactId>
<version>1.5.5</version>
</dependency>

```

Uso: Reservado (usando JavaFX Charts nativos para mejor integración)

4. JavaFX Swing 22.0.2 (Bridge)

```
<dependency>
<groupId>org.openjfx</groupId>
<artifactId>javafx-swing</artifactId>
<version>22.0.2</version>
</dependency>
```

Uso: SwingFXUtils.fromFXImage() para capturar gráficas JavaFX como imágenes PNG para embeber en PDF

■ Componentes Principales

1. FinancialReportData

Propósito: DTO principal que encapsula todos los datos del reporte financiero

Campos Principales:

- `ingresosTotales`: BigDecimal
- `devolucionesTotales`: BigDecimal
- `ingresosNetos`: BigDecimal
- `porcentajeRentabilidad`: BigDecimal
- `ticketPromedio`: BigDecimal
- `totalTransacciones`: Long
- `transaccionesAprobadas`: Long
- `transaccionesRevertidas`: Long
- `transaccionesRechazadas`: Long

Clases Anidadas:

- `DailyRevenue`: fecha, ingresos, devoluciones, neto
- `PaymentTypeDistribution`: tipoPago, monto, cantidad, porcentaje

2. FinancialReportService

Propósito: Servicio de lógica de negocio para cálculos financieros

Métodos Clave:

```
// Genera reporte completo para rango de fechas
FinancialReportData generateFinancialReport(LocalDate desde, LocalDate hasta)
// Calcula rentabilidad con división segura
```

```

BigDecimal calcularRentabilidad(BigDecimal ingresos, BigDecimal devoluciones)
// Agrupa transacciones por día con llenado de gaps
List<DailyRevenue> calcularIngresosDiarios()
// Calcula distribución porcentual por tipo de pago
List<PaymentTypeDistribution> calcularDistribucionPorTipo()
// Convierte PagoEnriquecidoDto a TransactionSummary
List<TransactionSummary> generarResumenTransacciones()

```

Dependencias:

- `PaymentApiClient`: Para listar pagos del backend
- `ObjectMapper`: Para parsear JSON de metadatos

3. FinancialDashboardView

Propósito: Vista premium del dashboard financiero

Componentes UI:

1. **Header:** Título "■ Reportes Económicos" + lblPeriodo dinámico
2. **Panel de Filtros:**
 - DatePickers (desde/hasta)
 - 4 botones rápidos (Hoy/Semana/Mes/Año)
 - btnActualizar, btnExportPDF, btnExportCSV
3. **Grid de Métricas (4x2):**
 - Fila 1: Ingresos Totales ■, Devoluciones ■, Ingresos Netos ■, Rentabilidad ■
 - Fila 2: Total Transacciones ■, Aprobadas ■, Revertidas ■■, Ticket Promedio ■
4. **Grid de Gráficas:**
 - LineChart: Evolución temporal (3 series)
 - BarChart: Comparativa simplificada
 - PieChart: Distribución por tipo con %
5. **Tabla Detallada:** 8 columnas con transacciones

Animaciones:

- ScaleTransition (1.0 → 1.03) en hover de cards
- FadeTransition (200ms) al actualizar métricas

Estilos:

- Gradientes azules corporativos
- Bordes coloreados según tipo de métrica
- Sombras dropshadow para profundidad
- Border-radius 12-16px para suavidad

4. PDFExportService

Propósito: Generación de PDFs profesionales con iText 7

Estructura del PDF:

[illegible]

13. PERFORMANCE

FASE 7.3 · Pruebas de carga y performance

Este documento describe cómo ejecutar pruebas de carga para EduFeed y verificar los objetivos de latencia p95 por endpoint.

Objetivos

- Concurrencia: 500 usuarios activos sin degradación
- Latencias (p95):
- POST /api/accesos/verificar < 2s
- Operaciones de pagos (<=> /api/pagos/*) < 1s
- Reportes (/api/reportes/*) < 3s

Preparación

1) Levantar base de datos

- VS Code Task: "DB: up"

2) Iniciar backend (perfil dev)

- VS Code Task: "Backend: run" (el puerto por defecto es 8080)
- Credenciales sembradas (por defecto): admin / Admin123\$

3) Variables útiles

- Base URL: http://localhost:8080
- Usuario Admin: configurable vía SEEDOPERATORUSERNAME / SEEDOPERATORPASSWORD

Opción A: Gatling (recomendada)

Se incluye un módulo opcional `edufeed-perf` con 3 simulaciones:

- AccessCheckSimulation: POST /api/accesos/verificar
- PaymentsSimulation: creación y listado de pagos (GET /api/pagos/estado/APROBADO)
- ReportsSimulation: GET /api/reportes/ingresos

Notas:

- El módulo `edufeed-perf` NO está agregado al build raíz; ejecútalo de forma independiente con `pl edufeed-perf`.
- Autenticación: cada simulación hace login y reutiliza el accessToken (Bearer).

Parámetros

- `-DbaseUrl=http://localhost:8080`
- `-Dusers=500` número de usuarios virtuales
- `-DrampSeconds=60` rampa de llegada
- `-DholdSeconds=300` fase de sostenimiento (usado para calcular p95 estable)

Ejemplos (PowerShell)

```
# AccessCheck, 500 usuarios
$env:JAVA_HOME='C:\Program Files\Java\jdk-24'
$base='http://localhost:8080'
$mvn="$env:USERPROFILE\tools\maven\apache-maven-3.9.9\bin\mvn.cmd"
& $mvn -pl edufeed-perf -am gatling:test `
-Dgatling.simulationClass=co.cellano.edufeed.perf.AccessCheckSimulation `
-DbaseUrl=$base -Dusers=500 -DrampSeconds=60 -DholdSeconds=300

# Payments
& $mvn -pl edufeed-perf -am gatling:test `
-Dgatling.simulationClass=co.cellano.edufeed.perf.PaymentsSimulation `
-DbaseUrl=$base -Dusers=500 -DrampSeconds=60 -DholdSeconds=300

# Reports
& $mvn -pl edufeed-perf -am gatling:test `
-Dgatling.simulationClass=co.cellano.edufeed.perf.ReportsSimulation `
-DbaseUrl=$base -Dusers=500 -DrampSeconds=60 -DholdSeconds=300
```

Asserts

- Cada simulación incluye asserts de p95 (percentile3) y 99%+ de éxito.
- Los reportes HTML quedan en `edufeed-perf/target/gatling/<simulacion>-<timestamp>/index.html`.

Opción B: JMeter

Si prefieres JMeter:

- Crea un Test Plan con:
- HTTP Request Defaults (Base URL)
- HTTP Header Manager (Content-Type: application/json)
- Login (POST /api/auth/login) + JSON Extractor (accessToken -> var)
- Header Manager hijo para "Authorization: Bearer \${accessToken}"
- Samplers según endpoint objetivo (ver arriba)

- Concurrency Thread Group: 500 usuarios, rampa 60s, hold 300s
- Backend Listener o Summary Report para p95

CLI (ejemplo):

```
jmeter -n -t edufeed-load.jmx -l results.jtl -e -o report
```

Optimización (índices SQL)

Se agregó `V6\_\_perf\_indexes.sql` (Flyway) con índices sugeridos para consultas frecuentes:

- pagos: *estadopago + creado*, *usuarioid + creado*, *tipopago + creado*, *referencia_externa*
- accesos: (*usuarioid, fechahora*), (*estado, fechahora*), (*modalidad, fechahora*)
- derechosuso: (*activo, usuarioid*), (*vigentedesde, vigentehasta*)

Aplica migraciones reiniciando el backend (Flyway valida y migra al arranque).

Interpretación de resultados

- Objetivo cumplido si los asserts pasan y el reporte HTML muestra p95 <= objetivo para cada endpoint medido.
- Si no se cumple:
- Revisa `Explain Analyze` de las consultas implicadas.
- Verifica que las migraciones de índices estén aplicadas.
- Aumenta el pool Hikari si hay espera por conexiones.
- Ajusta paginación en listados grandes.

Troubleshooting

- Puerto 8080 en uso: cierra procesos previos o cambia `server.port`.
- 401/403: confirma credenciales admin de `application.yml` o variables de entorno.
- Errores 5xx durante carga: revisa logs `logs/edufeed-backend.log` y métricas en `/actuador/metrics`.

14. CRITERIOS DE ACEPTACION

■ Criterios de Aceptación SMART - Sistema EduFeed

- > **Fecha de creación:** 20 de octubre de 2025
- > **Propósito:** Definir criterios medibles, específicos y verificables para cada requisito funcional y no funcional del sistema EduFeed.

■ Metodología SMART

- Cada criterio debe ser:
- **Specific (Específico):** Claro y sin ambigüedades
 - **Measurable (Medible):** Cuantificable con métricas concretas
 - **Achievable (Alcanzable):** Realista con los recursos disponibles
 - **Relevant (Relevante):** Alineado con los objetivos del proyecto
 - **Time-bound (Limitado en tiempo):** Con plazos definidos

■ RF-01: Registro de Usuarios

Criterios de Aceptación

ID	Criterio	Métrica	Objetivo
RF-01.1	Formulario completo de registro	Campos obligatorios validados	100% de campos con validación client-side y server-side
RF-01.2	Validación de documento único	Detección de duplicados	0 duplicados permitidos en BD
RF-01.3	Validación de formato email	Regex validación	100% emails con formato válido (RFC 5322)
RF-01.4	Validación de teléfono	Regex validación	100% teléfonos con formato +57\d{10}
RF-01.5	Tiempo de respuesta registro	Latencia	≤1 segundo (p95)
RF-01.6	Registro persistente	Integridad de datos	100% registros correctamente almacenados en BD

Casos de Prueba

Caso 1: Registro exitoso

- **Dado** un usuario nuevo con datos válidos
- **Cuando** se envía el formulario de registro
- **Entonces** el sistema crea el usuario, retorna código 201, y almacena correctamente en BD

Caso 2: Documento duplicado

- **Dado** un documento que ya existe en el sistema
- **Cuando** se intenta registrar con ese documento
- **Entonces** el sistema retorna 409 Conflict con mensaje "Documento ya registrado"

Caso 3: Email inválido

- **Dado** un email sin formato válido (ej: "usuario@.com")
- **Cuando** se envía el formulario
- **Entonces** el sistema retorna 400 Bad Request con detalle del error

■ RF-02: Validación Biométrica

Criterios de Aceptación

ID	Criterio	Métrica	Objetivo
RF-02.1	Tasa de acierto (huella)	FAR (False Accept Rate)	≤0.01%
RF-02.2	Tasa de rechazo (huella)	FRR (False Reject Rate)	≤5%
RF-02.3	Tasa de acierto (rostro)	Precisión	≥90% en condiciones controladas
RF-02.4	Tasa de acierto (voz)	Precisión	≥85% en ambiente con ruido moderado
RF-02.5	Latencia de verificación	Tiempo de respuesta	<2 segundos (p95)
RF-02.6	Captura de plantilla	Tiempo máximo	≤5 segundos por modalidad
RF-02.7	Almacenamiento seguro	Cifrado	100% plantillas cifradas con AES-256
RF-02.8	Verificación 1:1 (identificación)	Latencia	<1 segundo

Casos de Prueba

Caso 1: Enrolamiento de huella exitoso

- **Dado** un usuario sin plantilla biométrica
- **Cuando** se captura la huella en el dispositivo
- **Entonces** el sistema almacena la plantilla cifrada y retorna 201

Caso 2: Verificación exitosa de rostro

- **Dado** un usuario con plantilla facial enrolada
- **Cuando** se captura el rostro y se verifica
- **Entonces** el sistema retorna match=true en <2s

Caso 3: Rechazo por no coincidencia

- **Dado** una plantilla que no coincide con ninguna del sistema
- **Cuando** se intenta verificar
- **Entonces** el sistema retorna match=false sin error

■ RF-03: Control de Derecho Adquirido

Criterios de Aceptación

ID	Criterio	Métrica	Objetivo
RF-03.1	Validación en tiempo real	Latencia consulta	<500ms (p95)
RF-03.2	Precisión de vigencias	Exactitud temporal	100% derechos con vigencias correctas
RF-03.3	Consumo de paquetes	Decremento correcto	100% decrementos registrados atómicamente
RF-03.4	Detección de expiración	Validación diaria	100% derechos expirados detectados al día siguiente
RF-03.5	Concurrencia	Transacciones simultáneas	Soporte para ≥50 validaciones/segundo sin pérdida de datos

Casos de Prueba

Caso 1: Acceso con derecho vigente DIARIO

- **Dado** un usuario con pago DIARIO válido para hoy
- **Cuando** se verifica el acceso a las 12:00
- **Entonces** el sistema retorna permitido=true

Caso 2: Acceso con derecho expirado

- **Dado** un usuario con derecho vencido ayer
- **Cuando** se verifica el acceso
- **Entonces** el sistema retorna permitido=false, motivo="SIN_DERECHO"

Caso 3: Consumo de paquete

- **Dado** un usuario con paquete de 5 días, 3 consumidos
- **Cuando** se aprueba el acceso
- **Entonces** el sistema decrementa a 2 días restantes

■ RF-04: Orientación a Caja

Criterios de Aceptación

ID	Criterio	Métrica	Objetivo
RF-04.1	Visualización de mensaje	Tiempo de respuesta UI	<200ms
RF-04.2	Generación de código QR	Tiempo de generación	<500ms
RF-04.3	Legibilidad de instrucciones	Tamaño de fuente	≥16pt, contraste AAA (WCAG 2.1)
RF-04.4	Referencia única	Unicidad	100% referencias únicas (UUID o alfanumérico 6 dígitos)

Casos de Prueba

Caso 1: Usuario denegado sin derecho

- **Dado** un usuario sin derecho vigente
- **Cuando** se intenta el acceso
- **Entonces** se muestra pantalla de orientación con QR, referencia, e instrucciones claras

■ RF-05: Registro de Pagos

Criterios de Aceptación

ID	Criterio	Métrica	Objetivo
RF-05.1	Validación de montos	Monto mínimo	>0, máximo 1,000,000 COP
RF-05.2	Generación automática de derechos	Tiempo de procesamiento	<1 segundo tras aprobar pago
RF-05.3	Integridad transaccional	Atomicidad	100% pagos aprobados generan derecho o rollback
RF-05.4	Registro de método de pago	Opciones	EFFECTIVO, TARJETA, TRANSFERENCIA, OTRO
RF-05.5	Persistencia de recibo	Formato	PDF con datos completos, almacenado en carpeta recibos/

Casos de Prueba

Caso 1: Pago DIARIO

- **Dado** un monto de 8,000 COP tipo DIARIO
- **Cuando** se aprueba el pago
- **Entonces** se crea DerechoUso vigente para hoy

Caso 2: Pago MENSUAL

- **Dado** un monto de 120,000 COP tipo MENSUAL

- **Cuando** se aprueba el pago
- **Entonces** se crea DerechoUso vigente desde hoy hasta fin de mes

Caso 3: Pago PAQUETE

- **Dado** un monto de 35,000 COP, 5 días
- **Cuando** se aprueba el pago
- **Entonces** se crea PaquetePago con dias_restantes=5

■ RF-06: Reporte de Asistencia

Criterios de Aceptación

ID	Criterio	Métrica	Objetivo
RF-06.1	Generación de reporte	Latencia	<3 segundos para 1,000 registros
RF-06.2	Filtros funcionales	Tipos de filtro	Fecha, rango, tipo de derecho, usuario
RF-06.3	Exportación CSV	Formato válido	100% archivos con encoding UTF-8, delimitador coma
RF-06.4	Exportación Excel	Formato válido	100% archivos .xlsx con estilos (header bold)
RF-06.5	Exportación PDF	Diseño	Logo, tabla con datos, pie de página con fecha
RF-06.6	Paginación	Rendimiento	Soporte para ≥10,000 registros sin degradación

Casos de Prueba

Caso 1: Reporte de asistencias del día

- **Dado** filtro fecha=hoy
- **Cuando** se genera el reporte
- **Entonces** se muestran todas las asistencias del día en <3s

Caso 2: Exportación CSV

- **Dado** un reporte generado con 500 registros
- **Cuando** se exporta a CSV
- **Entonces** se descarga archivo válido con todas las filas

■ RF-07: Gestión de Usuarios

Criterios de Aceptación

ID	Criterio	Métrica	Objetivo
RF-07.1	CRUD completo	Endpoints	100% operaciones (Create, Read, Update, Delete) funcionales
RF-07.2	Soft delete	Auditoría	0 eliminaciones físicas, 100% usuarios marcados como inactivos
RF-07.3	Búsqueda por criterios	Latencia	<500ms para 10,000 usuarios
RF-07.4	Paginación	Tamaño de página	Configurable (10, 25, 50, 100)
RF-07.5	Validación de duplicados	Documento único	0 duplicados en UPDATE

Casos de Prueba

Caso 1: Listar usuarios con paginación

- **Dado** 500 usuarios en BD
- **Cuando** se solicita página 2, tamaño 25
- **Entonces** se retornan usuarios 26-50

Caso 2: Desactivar usuario

- **Dado** un usuario activo
- **Cuando** se ejecuta DELETE /api/users/{id}
- **Entonces** el usuario queda con activo=false, no se elimina físicamente

■ RF-08: Integración con Caja

Criterios de Aceptación

ID	Criterio	Métrica	Objetivo
RF-08.1	Webhook funcional	Disponibilidad	99.9% uptime
RF-08.2	Matching automático	Precisión	≥95% transacciones conciliadas automáticamente
RF-08.3	Reintentos con backoff	Estrategia	1s, 2s, 4s, 8s, fallo final
RF-08.4	Firma de webhook	Validación	100% requests con firma HMAC válida
RF-08.5	Conciliación manual	UI	Pantalla para vincular transacciones no match

Casos de Prueba

Caso 1: Webhook con referencia válida

- **Dado** una notificación de caja con referencia_externa que coincide
- **Cuando** se recibe el webhook
- **Entonces** se crea TransaccionCaja, se vincula Pago, conciliado=true

Caso 2: Webhook sin match

- **Dado** una notificación sin referencia coincidente
- **Cuando** se recibe el webhook
- **Entonces** se crea TransaccionCaja, conciliado=false, disponible para revisión manual

EduFeed

INFORMACION DEL PROYECTO

Nombre del Proyecto: EduFeed - Sistema de Control de Acceso para Restaurante Escolar

Version Actual: 3.0.0 (Noviembre 2025)

Repositorio: <https://github.com/Joan-Mora/EduFeed>

Rama Principal: main

Licencia: Privada

Estado: Produccion - Listo para despliegue

Ultima Actualizacion: 14 de November de 2025

Modulos del Sistema

- **edufeed-backend:** API REST con Spring Boot, endpoints de autenticacion, pagos, reportes
- **edufeed-desktop:** Aplicacion JavaFX para punto de acceso y caja
- **edufeed-biometric:** Libreria de interfaces y proveedores biometricos
- **edufeed-common:** DTOs y modelos compartidos entre modulos
- **edufeed-perf:** Pruebas de performance con JMeter

Fin del Documento

Este documento ha sido generado automaticamente a partir de la documentacion markdown del proyecto EduFeed. Para informacion actualizada, consulte el repositorio.

2025 EduFeed - Todos los derechos reservados